

Product document for a multi-tenant restaurant SaaS

Executive summary

This product should be built as a **B2B restaurant operating system** sold on recurring subscription, with four core surfaces: a restaurant CMS, a QR-launched customer PWA, an optional waiter/staff app, and a kitchen display. The strongest commercial and UX position is a **hybrid dine-in model**: the customer scans a table QR, enters a live table session, collaborates with friends in a shared bucket, and the restaurant can switch per branch between **self-service ordering** and **waiter-confirmed ordering**. A PWA is the right customer channel because it preserves web reach while adding installability, service-worker-driven offline behavior, and the browser push stack needed for service-request and order-status alerts. ¹

Architecturally, the best default is a **hybrid multitenant data model**: start with a shared PostgreSQL deployment in which every tenant-owned row carries a `tenant_id`, enforce isolation both in application code and with PostgreSQL row-level security, and keep a tenant catalog so that large or noisy tenants can later be promoted to dedicated databases or shards. That recommendation follows mainstream SaaS guidance: multitenant databases have the lowest per-tenant cost, but they sacrifice isolation and raise noisy-neighbor risk; database-per-tenant improves isolation; hybrid patterns let an operator move tenants between shared and isolated footprints as their needs change. PostgreSQL is the better default than MongoDB for this domain because restaurant operations are fundamentally relational and transactional, while PostgreSQL also gives you JSONB for flexible catalog metadata, declarative partitioning for high-write event tables, and built-in row security. MongoDB supports multi-document transactions, so it remains a viable secondary option for document-heavy modules, but it is not the best primary system of record for this workflow. ²

For collaboration and live operations, use **authoritative REST APIs for transactions** and **WebSockets for presence, bucket collaboration, staff alerts, and kitchen updates**. Socket.IO rooms map naturally to `tableSession`, `order`, `branch`, and `staff` scopes. If you scale past one realtime node, avoid relying on the classic Redis Pub/Sub adapter for critical collaboration state; Redis Pub/Sub is at-most-once, and Socket.IO's Redis adapter can stop propagating to other nodes during a Redis disconnect. For this product, the safer default is the Redis Streams adapter, which is specifically documented as handling temporary Redis disconnections and resuming stream delivery without losing packets. ³

The commercial model should be **base subscription plus add-ons**, not per-order “taxing” of restaurant revenue. The best default packaging is by branch and feature band, with optional add-ons for extra branches, premium analytics, SMS, branded domain, POS integrations, and dedicated tenant isolation. Recurring billing, customer self-service billing, invoices, trials, and usage-based add-ons are well supported by Stripe Billing and Stripe's customer portal, while Razorpay provides recurring subscriptions and webhook-driven subscription events for India-first deployments. ⁴

Decision area	Recommendation	Rationale
Customer channel	QR-based PWA with guest mode, optional install prompt, order tracking, and service request buzzer	Fast adoption and app-like capabilities without app-store friction. ¹
Tenancy model	Shared PostgreSQL first, hybrid promotion path later	Best early cost profile while preserving a route to tenant isolation for premium accounts. ⁵
Collaboration model	Shared bucket as mutable collaboration object; submitted order as immutable snapshot	Prevents kitchen and waiter confusion when multiple guests keep browsing after submit.
Realtime backbone	Socket.IO rooms plus Redis Streams adapter	Good fit for table/session fanout and safer cross-node recovery than classic Pub/Sub fanout. ⁶
Subscription platform	Default global path through Stripe ⁷ ; India-first path through Razorpay ⁸	Both support recurring billing; Stripe is stronger for global SaaS billing and customer portal, Razorpay is stronger for India-centric payments and subscriptions. ⁹
Launch infrastructure	Containerized services on a managed platform first, with portability to larger cloud footprints later	Faster time to market, lower ops burden, and easier later migration. ¹⁰

Product scope and feature roadmap

The product boundary should be explicit: this is first a **dine-in restaurant SaaS**, not a general food-delivery marketplace. The first release should optimize for restaurants that want to digitize menus, reduce waiter friction, support group ordering, and improve service responsiveness. That means the canonical flow is **table-bound ordering**, not delivery logistics. In practice, the CMS should support three service modes per branch: self-service, waiter-confirmed, and hybrid. Hybrid matters because many restaurants want customers to browse and build the order themselves while still preserving a human confirmation step before the kitchen sees the ticket. That switch must be branch-configurable and time-sensitive, not hard-coded globally.

Capability	MVP	Phase 2	Phase 3
Tenant onboarding, free trial, subscription activation	Yes	Expand entitlements	Enterprise procurement flows
Restaurant profile, branding, hours, tax, currency	Yes	Branch templates	White-label theme packs

Capability	MVP	Phase 2	Phase 3
Menu CMS: categories, subcategories, dishes, variants, add-ons	Yes	Bulk import/export	AI-assisted menu optimization
Availability windows and daypart scheduling	Yes	Advanced inventory tie-ins	Predictive stock-aware availability
Table management and QR generation	Yes	Reservations signal overlay	Dynamic QR campaigns
Guest QR menu and browsing filters	Yes	Personalization and favorites	Loyalty-aware recommendations
Shared table bucket ordering	Yes	Split-billing attribution	Cross-device continuity across visits
Waiter confirmation toggle	Yes	Waiter/table assignment rules	SLA- and staffing-aware automation
Service-request buzzer	Yes	Escalation rules and heuristics	Cross-channel orchestration
Kitchen display	Basic states	Station routing and prioritization	Multi-kitchen orchestration
Cashier and billing workflows	Basic	Partial/split payments	Complex settlement and franchise reporting
Promotions and coupons	Basic	Loyalty and wallet	Segmented lifecycle marketing
Analytics	Core operational dashboards	Cohort and retention analytics	Predictive analytics and benchmarking
Multi-branch operations	Limited	Centralized owner controls	Franchise and region admins
Integrations	Payment providers only	POS, printers, email/ SMS	Inventory, ERP, accounting, SSO

The packaging model should reward branch growth, not penalize restaurants simply for selling more. A recommended commercial structure is below.

Plan	Target customer	Suggested entitlement shape
Launch	Single-site cafe or small restaurant	One branch, up to a table band, QR menu, shared bucket, self-service or waiter-confirmed ordering, basic analytics, one kitchen display

Plan	Target customer	Suggested entitlement shape
Growth	Busy dine-in operator	Larger table band, multiple staff roles, kitchen routing, advanced analytics, promotions, split bill, branded domain, priority support
Enterprise	Chains and premium operators	Multi-branch governance, dedicated tenant isolation option, SSO, custom SLAs, integration bundles, data export, audit features

A good add-on strategy is to sell features that create obvious operational value: extra branches, advanced analytics, premium support, SMS bundles, white-label domain, POS connectors, printer connectors, reservation module, loyalty, and dedicated database isolation. If usage-based charging is needed, meter **operational extras** rather than core restaurant revenue. Examples include SMS volume, advanced export jobs, or additional analytics seats. Stripe supports usage-based billing through meters, and its hosted customer portal covers payment-method updates, invoice access, and subscription management; Razorpay supports recurring plans and webhook-driven subscription state changes. ¹¹

The role model should be explicit enough to power both UI and API authorization.

Role	Menu CMS	Live orders	Table map	Staff admin	Billing admin	Analytics	Settings
Platform super admin	Global	Global	Global	Global	Global	Global	Global
Restaurant owner/admin	Full	Full	Full	Full	Full	Full	Full
Branch manager	Branch	Branch	Branch	Limited	No	Branch	Limited
Waiter	No	Assigned and branch-level	Assigned	No	No	No	No
Kitchen staff	No	Ticket states only	No	No	No	No	No
Cashier	No	Billing-related	Table closure only	No	Settlement only	Limited	No
Customer guest	Browse only	Own session only	Own table only	No	Own payment only	No	No

Role	Menu CMS	Live orders	Table map	Staff admin	Billing admin	Analytics	Settings
Customer account holder	Browse plus history/favorites	Own session only	Own table only	No	Own payment only	Limited history	No

Roles, journeys, and interface design

The customer-facing experience should be optimized as a **mobile-first PWA**. That allows you to launch from QR with no install requirement, but still offer installability for repeat visitors, and use service workers for offline caching, background capabilities, and push integration where browser support allows it. Service workers are explicitly designed to sit between the page and the network and enable offline experiences, background sync, and push notification access. ¹

flowchart TD

A[Platform Admin] --> A1[Tenants]

A --> A2[Plans]

A --> A3[Billing Ops]

A --> A4[Support and Flags]

B[Restaurant CMS] --> B1[Dashboard]

B --> B2[Orders]

B --> B3[Tables and QR]

B --> B4[Menu]

B --> B5[Service Requests]

B --> B6[Kitchen]

B --> B7[Staff and Roles]

B --> B8[Analytics]

B --> B9[Subscription]

B --> B10[Settings]

C[Customer PWA] --> C1[Landing]

C --> C2[Menu and Search]

C --> C3[Item Detail]

C --> C4[Shared Bucket]

C --> C5[Order Status]

C --> C6[Service Request]

C --> C7[Bill and Payment]

C --> C8[Feedback]

D[Waiter App] --> D1[Live Tables]

D --> D2[Confirm Orders]

D --> D3[Service Queue]

D --> D4[Bill Requests]

E[Kitchen Display] --> E1[New]
 E --> E2[Preparing]
 E --> E3[Ready]
 E --> E4[Served or Archived]

The CMS should be designed around **operational speed**, not decorative administration. The goal is to let a manager resolve a live issue in two taps. That suggests the following page set and wireframe behavior.

CMS page	Primary goal	Wireframe description
Dashboard	Instant operational awareness	Top header with branch selector, date, and mode switch. First row of KPI cards: live orders, active tables, pending requests, revenue today. Second row: order timeline chart, top dishes, slow tickets, recent alerts. Right rail: unresolved service requests and payment issues.
Orders	Manage all live and historical orders	Left filter column for status, table, waiter, kitchen station, date. Center list shows cards or rows grouped by status. Right-side drawer opens order detail, audit timeline, modifier breakdown, participant attribution, and payment state. Sticky action bar for confirm, reject, mark preparing, ready, served, and refund.
Tables map	Visual floor control	Floor tabs at top, drag-and-drop tables in canvas, color-coded states for free, occupied, waiting confirmation, preparing, ready, bill requested, cleaning. Clicking a table opens session drawer with participants, bucket snapshot, past orders, request queue, and bill actions.
QR manager	Issue and manage QR surfaces	Table listing with columns for floor, table number, QR status, last scan, active session, reprint, regenerate. Batch actions for export PDF and print pack.
Categories and subcategories	Maintain menu structure	Two-pane layout: categories left, subcategories center, preview right. Drag to reorder. Inline visibility toggles and schedule badges.
Menu item editor	Create and edit dishes	Form split into tabs: basics, pricing and variants, add-ons, tags and allergens, scheduling, media, branch overrides. Right rail shows mobile customer preview. Sticky save and publish controls.
Availability and schedule rules	Control when items can appear	Calendar-like rule builder with presets for breakfast/lunch/dinner, weekday ranges, blackout dates, sold-out toggle, and "inherit from branch hours" option.

CMS page	Primary goal	Wireframe description
Service requests	Resolve customer assistance quickly	Queue view with request type, table, age, assigned waiter, SLA color, and quick actions. Default sorting by expired or aging first.
Kitchen board	Prepare orders efficiently	Kanban columns for new, accepted, preparing, ready, delayed. Cards show table, order age, item count, modifiers, allergy flags, and rush marker. Bulk move by ticket or by item group.
Staff and roles	Control access and assignments	Staff table with branch, role, status, shift window, last active, MFA status. Role editor opens permission matrix and branch scope.
Analytics	Review performance and trends	Date, branch, and service-mode filters on top. Charts for conversion, average basket, table turnover, request response time, top modifiers, category mix, and menu availability impact. Export button for CSV and PDF.
Subscription and billing	Let the restaurant manage its own plan	Summary card for current plan, renewal date, invoice history, payment method, add-ons, and usage meters. Upgrade CTA opens hosted billing flow.

The **customer journey** must prioritize zero-friction entry. Recommended interaction model: the first guest scans QR, the system opens a `table_session`, asks for a lightweight alias, and shows the menu immediately. Friends joining later can scan the same QR or enter a small session code to join the live bucket. Every bucket item should persist both a **who added it** marker and an **item snapshot** so downstream order history remains stable even if the menu later changes.

```
sequenceDiagram
    participant C1 as Customer A
    participant C2 as Customer B
    participant PWA as Customer PWA
    participant API as API
    participant RT as Realtime
    participant W as Waiter App
    participant K as Kitchen Display

    C1->>PWA: Scan table QR
    PWA->>API: POST join table session
    API-->>PWA: session token + table context + bucket
    PWA->>RT: subscribe tableSession room

    C2->>PWA: Join same table
    PWA->>API: POST join table session
    API-->>PWA: same bucket + participant identity
```

```

PWA->>RT: subscribe tableSession room

C1->>PWA: Add item
PWA->>API: POST bucket item
API->>RT: emit bucket.item_added
RT-->>C1: bucket delta
RT-->>C2: bucket delta

C2->>PWA: Add item and note
PWA->>API: POST bucket item
API->>RT: emit bucket.item_added
RT-->>C1: bucket delta
RT-->>C2: bucket delta

C1->>PWA: Submit bucket
PWA->>API: POST bucket submit
API->>W: waiter confirmation needed?
alt Waiter confirmation
    W->>API: confirm order
end
API->>K: create kitchen ticket
API->>RT: emit order.created and order.status
RT-->>C1: order accepted
RT-->>C2: order accepted

```

The **service-request buzzer** should be its own first-class workflow, not just a chat button. Requests such as “water,” “bill,” “cutlery,” and “call waiter” should create structured events with SLA timers, waiter assignment, and escalation.

```

sequenceDiagram
    participant C as Customer
    participant PWA as Customer PWA
    participant API as API
    participant RT as Realtime
    participant W as Waiter App
    participant M as Manager Dashboard

    C->>PWA: Tap Call Waiter
    PWA->>API: POST service request
    API->>RT: emit service_request.created
    RT-->>W: new request for assigned branch/table
    Note over W: waiter countdown starts

    alt Waiter resolves in SLA
        W->>API: PATCH service request resolved
        API->>RT: emit service_request.resolved
    end

```



```

    RT-->>PWA: request resolved
else SLA expires
    API-->>RT: emit service_request.escalated
    RT-->>M: escalated request
    M-->>API: assign alternate staff
end

```

Domain model and API contract

The data model should separate **catalog**, **session collaboration**, **submitted commerce objects**, and **operational events**. The most important product invariant is that the **shared bucket is mutable, but the order is immutable once submitted**. After submission, changes should happen through explicit follow-on order amendments or new order versions, not by editing the prior submitted order in place. This is what keeps waiter confirmation, kitchen tickets, and billing coherent in a multi-user session.

The database of record should be PostgreSQL. The justification is straightforward: the domain requires strong consistency for orders, bills, and permissions; menu data still benefits from some flexibility, which PostgreSQL's `jsonb` supports; and high-write tables such as orders, notifications, and audit logs can be partitioned as data volume grows. PostgreSQL's row-level security provides a second layer of tenant isolation. MongoDB's support for distributed transactions is real, but the average restaurant SaaS workload still maps more naturally to relational joins, foreign keys, and transactional integrity. ¹²

```

erDiagram
    TENANT ||--o{ BRANCH : has
    TENANT ||--o{ MEMBERSHIP : has
    USER ||--o{ MEMBERSHIP : belongs_to
    ROLE ||--o{ MEMBERSHIP : grants

    TENANT ||--o{ SUBSCRIPTION : billed_as
    SUBSCRIPTION ||--o{ INVOICE : generates

    TENANT ||--o{ MENU_CATEGORY : owns
    MENU_CATEGORY ||--o{ MENU_SUBCATEGORY : groups
    MENU_SUBCATEGORY ||--o{ MENU_ITEM : contains
    MENU_ITEM ||--o{ ITEM_VARIANT : offers
    MENU_ITEM ||--o{ ADDON_GROUP : has
    ADDON_GROUP ||--o{ ADDON_OPTION : contains
    MENU_ITEM ||--o{ ITEM_SCHEDULE : follows

    BRANCH ||--o{ FLOOR : has
    FLOOR ||--o{ DINING_TABLE : contains
    DINING_TABLE ||--o{ QR_CODE : maps
    DINING_TABLE ||--o{ TABLE_SESSION : opens
    TABLE_SESSION ||--o{ TABLE_PARTICIPANT : includes
    TABLE_SESSION ||--|| BUCKET : owns
    BUCKET ||--o{ BUCKET_ITEM : contains

```

```

TABLE_SESSION ||--o{ ORDER : generates
ORDER ||--o{ ORDER_ITEM : contains
ORDER_ITEM ||--o{ ORDER_ITEM_ADDON : has
ORDER ||--o{ KITCHEN_TICKET : emits
DINING_TABLE ||--o{ SERVICE_REQUEST : raises
ORDER ||--o{ PAYMENT : settles

TENANT ||--o{ AUDIT_LOG : records
USER ||--o{ AUDIT_LOG : performs

```

The core schema can be expressed as follows.

Table	Key columns	Notes
tenants	id, slug, legal_name, status, default_currency, default_timezone	One commercial customer account; owns branches and entitlements.
branches	id, tenant_id, name, service_mode, address_json, hours_json	Branch-level operational settings; service_mode = self_service, waiter_confirmed, hybrid.
users	id, email, phone, password_hash, mfa_state, status	Staff and platform accounts; customer accounts can be separate or optional.
roles	id, scope, name	Platform standard roles plus tenant-custom roles.
memberships	id, tenant_id, branch_id, user_id, role_id	Grants scoped access to tenant and optional branch.
subscriptions	id, tenant_id, provider, provider_customer_id, provider_subscription_id, plan_code, status, trial_ends_at, renews_at	SaaS billing state; provider = stripe or razorpay.
invoices	id, subscription_id, provider_invoice_id, amount, currency, status, invoice_url	Reference to provider invoice records.
Table	Key columns	Notes
menu_categories	id, tenant_id, branch_id nullable, name, sort_order, is_visible	Branch nullable means tenant-default category.

Table	Key columns	Notes
<code>menu_subcategories</code>	<code>id</code> , <code>category_id</code> , <code>name</code> , <code>sort_order</code> , <code>is_visible</code>	Nested grouping under category.
<code>menu_items</code>	<code>id</code> , <code>tenant_id</code> , <code>subcategory_id</code> , <code>name</code> , <code>description</code> , <code>image_url</code> , <code>dietary_flags</code> , <code>allergens_json</code> , <code>base_price</code> , <code>is_active</code>	Core dish record.
<code>item_variants</code>	<code>id</code> , <code>item_id</code> , <code>name</code> , <code>price_delta</code> , <code>sku</code> , <code>is_default</code>	Size, portion, or style variants.
<code>addon_groups</code>	<code>id</code> , <code>item_id</code> , <code>name</code> , <code>selection_mode</code> , <code>min_select</code> , <code>max_select</code>	Modifier groups such as toppings or sides.
<code>addon_options</code>	<code>id</code> , <code>addon_group_id</code> , <code>name</code> , <code>price_delta</code> , <code>is_active</code>	Concrete add-on choices.
<code>item_schedules</code>	<code>id</code> , <code>item_id</code> , <code>days_mask</code> , <code>start_time</code> , <code>end_time</code> , <code>start_date</code> nullable, <code>end_date</code> nullable	Daypart and date-window availability.
<code>item_branch_overrides</code>	<code>id</code> , <code>item_id</code> , <code>branch_id</code> , <code>is_available</code> , <code>price_override</code> nullable	Branch-specific stock or pricing overrides.

Table	Key columns	Notes
<code>floors</code>	<code>id</code> , <code>branch_id</code> , <code>name</code> , <code>sort_order</code>	Dining area or floor map.
<code>dining_tables</code>	<code>id</code> , <code>branch_id</code> , <code>floor_id</code> , <code>table_no</code> , <code>capacity</code> , <code>status</code>	<code>status</code> tracks availability and occupancy.
<code>qr_codes</code>	<code>id</code> , <code>table_id</code> , <code>token</code> , <code>version</code> , <code>is_active</code> , <code>last_scanned_at</code>	Rotatable QR mapping.
<code>table_sessions</code>	<code>id</code> , <code>tenant_id</code> , <code>branch_id</code> , <code>table_id</code> , <code>opened_at</code> , <code>closed_at</code> nullable, <code>status</code> , <code>host_participant_id</code> nullable	One active session per physical table by rule.
<code>table_participants</code>	<code>id</code> , <code>table_session_id</code> , <code>guest_alias</code> , <code>guest_token_hash</code> , <code>joined_at</code> , <code>last_seen_at</code> , <code>is_active</code>	Lightweight guest presence records.

Table	Key columns	Notes
<code>buckets</code>	<code>id</code> , <code>table_session_id</code> , <code>state</code> , <code>submitted_at</code> nullable, <code>locked_at</code> nullable, <code>submitted_by_participant_id</code> nullable	Mutable collaboration object before submit.
<code>bucket_items</code>	<code>id</code> , <code>bucket_id</code> , <code>participant_id</code> , <code>item_snapshot_json</code> , <code>variant_snapshot_json</code> , <code>qty</code> , <code>notes</code> , <code>line_total</code>	Snapshot the catalog state at add time.
<code>orders</code>	<code>id</code> , <code>tenant_id</code> , <code>branch_id</code> , <code>table_session_id</code> , <code>table_id</code> , <code>order_no</code> , <code>service_mode</code> , <code>status</code> , <code>source</code> , <code>submitted_at</code> , <code>confirmed_by_user_id</code> nullable, <code>subtotal</code> , <code>tax_total</code> , <code>grand_total</code>	Immutable submitted order header.
<code>order_items</code>	<code>id</code> , <code>order_id</code> , <code>participant_id</code> nullable, <code>item_snapshot_json</code> , <code>qty</code> , <code>unit_price</code> , <code>notes</code> , <code>kitchen_status</code>	Immutable dish lines.
<code>order_item_addons</code>	<code>id</code> , <code>order_item_id</code> , <code>addon_snapshot_json</code> , <code>unit_price</code>	Immutable modifier lines.
<code>kitchen_tickets</code>	<code>id</code> , <code>order_id</code> , <code>station</code> , <code>status</code> , <code>started_at</code> nullable, <code>ready_at</code> nullable	Ticket slicing for kitchen operations.
<code>service_requests</code>	<code>id</code> , <code>tenant_id</code> , <code>branch_id</code> , <code>table_session_id</code> , <code>table_id</code> , <code>request_type</code> , <code>message</code> nullable, <code>status</code> , <code>assigned_user_id</code> nullable, <code>requested_at</code> , <code>resolved_at</code> nullable	Structured service workflow.
<code>payments</code>	<code>id</code> , <code>order_id</code> , <code>provider</code> , <code>provider_payment_id</code> , <code>amount</code> , <code>currency</code> , <code>method</code> , <code>status</code> , <code>captured_at</code> nullable	Dine-in payment state.
<code>audit_logs</code>	<code>id</code> , <code>tenant_id</code> , <code>actor_user_id</code> nullable, <code>actor_guest_id</code> nullable, <code>entity_type</code> , <code>entity_id</code> , <code>action</code> , <code>before_json</code> , <code>after_json</code> , <code>created_at</code>	Required for disputes and operational traceability.
<code>web_push_subscriptions</code>	<code>id</code> , <code>user_id</code> , <code>endpoint_hash</code> , <code>subscription_json</code> , <code>platform</code> , <code>last_seen_at</code>	Staff-device push routing.

The authentication model should use **different trust levels for different actors**. Staff and admins should authenticate with email plus password or magic link, with MFA required for high-privilege roles; customers should get a short-lived guest session token tied to tenant, branch, table, and participant alias rather than being forced to register. For API design, every request that dereferences a business object must perform both **tenant scoping** and **object-level authorization**, because broken object level authorization is among the most common API failures. All API traffic must be HTTPS-only, and privileged APIs should support stronger controls such as device binding or step-up MFA. ¹³

A recommended HTTP surface is below.

Method	Path	Actor	Purpose	Auth
POST	/v1/guest-sessions	Customer	Create anonymous guest identity	Public
POST	/v1/table-sessions/ join	Customer	Join or resume table session from QR	Guest token
GET	/v1/menu	Customer	Get filtered menu for branch plus table session	Guest token
POST	/v1/buckets/ {bucketId}/items	Customer	Add item to shared bucket	Guest token
PATCH	/v1/buckets/ {bucketId}/items/ {itemId}	Customer	Edit quantity or notes	Guest token
POST	/v1/buckets/ {bucketId}/submit	Customer	Submit bucket as order candidate	Guest token plus idempotency
POST	/v1/service-requests	Customer	Create structured help request	Guest token
POST	/v1/staff/auth/ login	Staff	Authenticate staff	Public
GET	/v1/staff/orders/ live	Waiter/ Manager	View branch live queue	Staff JWT
POST	/v1/staff/orders/ {orderId}/confirm	Waiter/ Manager	Confirm waiter-confirmed order	Staff JWT
PATCH	/v1/kitchen/tickets/ {ticketId}	Kitchen	Update prep state	Staff JWT
GET	/v1/cms/menu/items	Admin/ Manager	Manage menu catalog	Staff JWT

Method	Path	Actor	Purpose	Auth
POST	/v1/cms/menu/items	Admin/ Manager	Create dish	Staff JWT
PATCH	/v1/cms/menu/items/ {itemId}	Admin/ Manager	Update item, schedule, visibility	Staff JWT
GET	/v1/cms/tables	Admin/ Manager	Table/Qr management	Staff JWT
POST	/v1/billing/ checkout-session	Owner/ Admin	Start SaaS checkout	Owner JWT
POST	/v1/webhooks/stripe	Provider	Billing/payment events	Signature verified
POST	/v1/webhooks/ razorpay	Provider	Billing/payment events	Signature verified

Examples:

```
POST /v1/table-sessions/join
{
  "qr_token": "tbl_8w9x...signed",
  "guest_alias": "Ava"
}
```

```
200 OK
{
  "table_session": {
    "id": "tsn_01JZ...",
    "branch_id": "br_01...",
    "table_id": "tbl_12",
    "table_no": "12",
    "service_mode": "waiter_confirmed"
  },
  "participant": {
    "id": "ptc_01...",
    "guest_alias": "Ava"
  },
  "bucket": {
    "id": "bkt_01...",
    "state": "open",
    "items": []
  },
  "guest_token": "eyJhbGciOi..."
}
```

```
POST /v1/buckets/bkt01/items
{
  "item_id": "itm_paneer_tikka",
  "variant_id": "var_full",
  "qty": 2,
  "notes": "Less spicy",
  "addons": [
    { "addon_option_id": "add_extra_onion", "qty": 1 }
  ]
}
```

```
POST /v1/buckets/bkt01/submit
{
  "client_submit_id": "submit-01JZ-order",
  "payment_preference": "pay_later"
}
```

```
202 Accepted
{
  "order_id": "ord_01...",
  "status": "pending_confirmation",
  "waiter_confirmation_required": true
}
```

```
POST /v1/service-requests
{
  "table_session_id": "tsn_01JZ...",
  "request_type": "WATER",
  "message": null
}
```

```
201 Created
{
  "service_request_id": "srv_01...",
  "status": "open",
  "eta_hint_seconds": 120
}
```

The realtime contract should be small, explicit, and versioned.

Room	Event	Produced by	Consumed by	Payload shape
<code>tableSession:{id}</code>	<code>participant.joined</code>	API	Customer PWA	participant id, alias, joined_at
<code>tableSession:{id}</code>	<code>bucket.item_added</code>	API	Customer PWA	bucket item summary, totals, actor
<code>tableSession:{id}</code>	<code>bucket.item_updated</code>	API	Customer PWA	item delta, totals
<code>tableSession:{id}</code>	<code>bucket.locked</code>	API	Customer PWA	bucket state, submitter
<code>tableSession:{id}</code>	<code>order.status_updated</code>	API/ Kitchen	Customer PWA	order id, old/new status
<code>branch:{id}</code>	<code>order.created</code>	API	Waiter/ Kitchen	order header, table, age
<code>branch:{id}</code>	<code>service_request.created</code>	API	Waiter/ Manager	request type, table, SLA due
<code>staff:{id}</code>	<code>service_request.assigned</code>	API	Waiter	request payload plus escalation tier
<code>order:{id}</code>	<code>ticket.updated</code>	Kitchen	Waiter/ Customer	station, state transition

Two implementation rules matter. First, all mutating order and billing calls must carry an **idempotency key**, especially for payment initiation and bucket submission, to prevent duplicate charges or duplicate orders under retry conditions. Second, billing webhooks must be authenticated via provider signatures and handled asynchronously; entitlements should never be granted from a front-end “payment succeeded” page alone. Stripe documents idempotent requests and recommends webhook-signature verification; Razorpay’s webhook guidance explicitly calls out validation, idempotency, and raw-body signature handling.

14

Platform architecture and operations

The recommended launch architecture is **containerized, modular, and cloud-portable**. The application should be split into a web/API tier, a dedicated realtime tier, and background workers for billing, notifications, report generation, QR regeneration, and menu publication jobs. That keeps the order write path simple and lowers the chance that long-running jobs interfere with live dining operations. The approach also aligns with the broader SaaS guidance from AWS and Microsoft that emphasizes explicit tenancy decisions and cloud architectures that can be reasoned about for reliability, security, and cost.

15


```

flowchart LR
    subgraph Clients
        SA[Platform Admin]
        CMS[Restaurant CMS]
        PWA[Customer PWA]
        WA[Waiter App]
        KDS[Kitchen Display]
    end

    EDGE[CDN / WAF / TLS]
    API[API and Auth Service]
    RT[Realtime Gateway]
    JOBS[Background Workers]
    DB[(PostgreSQL)]
    CACHE[(Redis or Valkey)]
    OBJ[(Object Storage)]
    PSP[Payment Providers]
    OBS[Logs, Metrics, Traces]

    SA --> EDGE
    CMS --> EDGE
    PWA --> EDGE
    WA --> EDGE
    KDS --> EDGE

    EDGE --> API
    EDGE --> RT

    API --> DB
    API --> CACHE
    RT --> CACHE
    JOBS --> DB
    JOBS --> CACHE
    API --> OBJ
    JOBS --> OBJ

    API --> PSP
    JOBS --> PSP

    API --> OBS
    RT --> OBS
    JOBS --> OBS

```

A pragmatic launch path is to run the application on a managed platform such as [Render](#) ¹⁶, which publishes first-party support for full-stack services, background workers, WebSockets, managed PostgreSQL, Redis-compatible key-value services, and OpenTelemetry metrics streaming. If you prefer a

specialized frontend deployment model, Vercel ¹⁷ is strong for web frontends and includes automatic CI/CD, team collaboration, and integrated observability options. The important design choice is not the provider itself but that the services remain containerized and stateless so you can later migrate them to a larger cloud footprint without rewriting application logic. ¹⁸

Layer	Recommendation	Why this fits the product
Frontend	Next.js or React with TypeScript	Reuse UI primitives across CMS, PWA, waiter, and kitchen surfaces
Backend	NestJS or Fastify-based Node service	Type-safe domain boundaries, strong ecosystem for payment, websocket, and queue integrations
Database	PostgreSQL	Best fit for relational restaurant operations, transactional billing, RLS, JSONB, partitioning
ORM and migrations	Prisma or SQL-first migrations	Good DX plus schema discipline
Realtime	Socket.IO	Event model maps well to rooms per table, order, and branch
Cache and broker	Redis or Valkey	Session/cache, rate limiting, websocket fanout, background coordination
Jobs	BullMQ or equivalent worker system	Notification retries, webhooks, PDF exports, billing sync
Storage	Object storage for dish images, logos, QR assets	Keeps media off the primary database
Search	Postgres full-text initially	Menu search workload is moderate and can stay close to source of truth
Auth	JWT plus refresh rotation for staff, guest tokens for customers	Different trust levels for staff vs ephemeral diners
Analytics	OLTP-first dashboards, warehouse later	Avoid premature data-platform complexity

The third-party service choices should be explicit.

Category	Default choice	Strong alternatives	Best use case	Evidence
Payments	Stripe ⁷	Razorpay ⁸	Stripe is better when the business needs global SaaS billing, usage-based pricing, hosted portal, and broad payment-method coverage; Razorpay is better when the merchant base is concentrated in India and needs local rails and subscriptions.	Stripe documents subscriptions, usage-based meters, webhooks, a hosted customer portal, and standard payments pricing; Razorpay documents subscriptions, REST APIs, webhook flows, and standard gateway pricing. ¹⁹
Object storage	Amazon Web Services ²⁰ S3	Cloudflare ²¹ R2, Cloudinary ²²	S3 is the safest general-purpose default for scale and ecosystem depth; R2 is compelling when egress economics matter; Cloudinary is useful when image transformation and delivery optimization are product requirements rather than simple file hosting.	S3 emphasizes scalability, availability, security, and performance; R2 publishes pricing with free egress and request classes; Cloudinary positions itself around transformation, optimization, and managed media delivery. ²³
Realtime	Self-hosted Socket.IO plus Redis Streams	Aby ²⁴ , Pusher ²⁵	Self-hosting is best when you want protocol control, lower marginal cost, and tight coupling to order logic; Aby is strongest when you want globally managed pub/sub and presence at scale; Pusher is attractive for simpler hosted pub/sub and presence channels.	Socket.IO documents rooms and Redis Streams fanout; Aby documents pub/sub, presence, and pricing; Pusher documents realtime channels, presence, and quota-based pricing. ²⁶

For CI/CD, use **Git-based automation** with GitHub ²⁷ Actions to build, test, and deploy on every pull request and merge. Docker images should use multi-stage builds so runtime artifacts remain small and reproducible. For observability, instrument everything with OpenTelemetry and push exceptions and traces into Sentry ²⁸ or a comparable backend. That gives you structured traces across API, realtime, jobs, and payment webhook processing. ²⁹

Testing should follow the system's boundaries. Unit tests should cover pricing, discount logic, tax calculation, state transitions, permission checks, and tenant scoping. Integration tests should run against a real PostgreSQL instance for order submission, bucket-to-order conversion, and webhook processing. End-to-end tests should cover the four critical user surfaces with browser automation. Realtime tests should explicitly simulate concurrent guests editing one bucket and Redis restarts. Accessibility and performance

regressions should be checked continuously; Lighthouse is built to audit web-page quality, and WCAG 2.2 remains the right accessibility target. ³⁰

Data protection and recoverability also need to be designed, not bolted on. PostgreSQL logical exports via `pg_dump` and selective restores via `pg_restore` are appropriate for tenant-level archival and restore workflows, while managed database backups should provide point-in-time recovery for operational safety. If the product eventually moves to AWS, RDS supports automated backups and point-in-time restore, and cross-region backup replication is available for stronger disaster recovery. For major migrations, a managed migration path such as AWS DMS can reduce downtime. ³¹

Security, compliance, and billing

Security should be designed around **tenant isolation, privilege separation, and payment-scope minimization**. The minimum posture is: tenant-aware authorization on every request, HTTPS-only APIs, MFA for privileged users, centralized secrets management, and structured security logging. Those recommendations are directly aligned with guidance from OWASP ³²: API object-level authorization is a leading failure mode, REST services should be HTTPS-only, MFA is recommended for high-privileged users, secrets should be centrally managed and rotated, and logging should be designed deliberately rather than added ad hoc. ³³

A practical control set for this product is as follows. Use row-level security and tenant-bound query scopes for every tenant-owned table. Keep platform users and restaurant users on separate authorization scopes, even if they share an identity provider. Sign QR tokens so table context cannot be forged. Bind guest session tokens to `tenant_id`, `branch_id`, `table_session_id`, and `participant_id`, and reject reuse after session close. Store payment provider identifiers, never raw card data. Verify webhook signatures before any side effect, and make webhook handlers idempotent. Write audit logs for menu changes, role changes, subscription changes, order state transitions, and refunds.

Payment compliance requires care because the product mixes SaaS billing and restaurant payment collection. If online customer payments are implemented through hosted checkout components, the platform can avoid storing or transmitting electronic card data directly. PCI SAQ A is intended for merchants whose cardholder-data functions are completely outsourced to validated third parties and who do not store, process, or transmit account data electronically; the PCI Security Standards Council also states that SAQ A is **not** applicable to face-to-face channels. In practical terms, the safest design is: use hosted payment pages for customer PWA card payments and keep any future card-present terminal workflows isolated as a separate compliance stream. ³⁴

Privacy and accessibility should be planned before enterprise sales begin. If the product serves EU restaurants or diners, personal-data processing may fall under GDPR obligations, and the European Commission's guidance emphasizes protected personal data, rights handling, and cross-border transfer safeguards. Accessibility should target WCAG 2.2 AA across CMS, PWA, waiter, and kitchen interfaces; WCAG 2.2 is the current W3C recommendation and is explicitly written to make content more accessible across web content and devices. A future enterprise program may also require SOC 2; the AICPA defines SOC 2 as a controls report relevant to security, availability, processing integrity, confidentiality, or privacy. ³⁵

The billing lifecycle should be explicit in the product model.

Lifecycle state	Trigger	Entitlement behavior
<code>trialing</code>	Tenant created and trial started	All contracted features on, usage meters active, banners visible
<code>active</code>	First successful subscription activation or invoice payment	Full entitlements
<code>grace_period</code>	Renewal failed but retry window open	Restaurant keeps core ops; billing banners and owner notifications escalate
<code>past_due</code>	Retry window exceeded	Non-destructive restrictions, such as blocking feature expansion and hiding premium analytics
<code>suspended</code>	Manual suspension or extended non-payment	Customer PWA may remain browse-only; order-taking disabled
<code>cancelled</code>	Subscription cancelled and term ended	Preserve data retention/export window, then archive per policy

The restaurant owner should be able to self-serve billing operations without support tickets. Stripe's customer portal explicitly supports payment-method updates, invoice access, billing updates, subscription changes, and cancellation flows, which makes it a strong default when Stripe is the provider. For provider-side correctness, use provider webhooks as the source of truth for state transitions, not redirect-page callbacks. ³⁶

Economics, backlog, and rollout roadmap

Scalability in this product is driven more by **many small tenants with bursty local peaks** than by one giant always-hot workload. That is exactly why the hybrid tenancy path matters. Shared storage keeps early cost down; partitioning and hybrid tenant promotion absorb growth later. As order, event, and audit tables grow, use PostgreSQL partitioning for the write-heavy fact tables, and promote outlier tenants to isolated databases when their workload or compliance demands justify it. ³⁷

Rough infrastructure economics can be estimated with current published prices from managed platforms. The table below assumes a single region, low media volume, and no heavy SMS spend.

Scenario	Assumed stack	Rough monthly infra	Pricing basis
Pilot launch	One professional workspace, one standard API service, one standard worker or realtime service, basic managed Postgres, starter Redis-compatible cache, small object storage footprint	~\$100–\$150	Render professional workspace starts at \$19 per user/month plus compute; standard services are \$25 each; basic 1 GB Postgres is \$19/month; starter key-value is \$10/month; R2 storage is \$0.015/GB-month with free egress. ³⁸

Scenario	Assumed stack	Rough monthly infra	Pricing basis
Early production	Add another service or separate frontend hosting, move to higher managed Postgres tier, standard cache, modest observability	~\$220– \$400	Render Pro-4gb Postgres is \$55/month, standard key-value is \$32/month, additional standard services remain \$25/month, and Vercel Pro is \$20/month plus usage. ³⁹
Higher-availability regional production	Multiple app services, larger Postgres, larger cache, more bandwidth, observability add-ons, replicated environments	~\$600– \$900+	Based on published managed-platform service, database, cache, and frontend pricing, with uplift for higher tiers and additional services. ⁴⁰

Payment costs sit on top of infrastructure. Stripe publishes 2.9% + 30¢ for successful domestic card transactions on its standard online-card pricing and 0.7% of billing volume for Stripe Billing pay-as-you-go usage. Razorpay currently publishes 2% + GST per transaction under its standard gateway pricing and states there are no setup or annual maintenance charges under that model. ⁴¹

If the product later graduates to a more self-managed AWS footprint, costs will climb because you add more explicit HA and observability components. AWS documents that Fargate pricing is based on requested CPU, memory, and storage per second; RDS pricing depends on instance hours, storage, backups, and transfer; and CloudWatch pricing varies with metrics and log ingestion. In other words, a hardened AWS posture can absolutely be the right long-term destination, but it is rarely the cheapest way to validate a restaurant SaaS before product-market fit. ⁴²

A recommended prioritized backlog is below.

Priority	Epic	Outcome
P0	Tenant onboarding and billing core	Restaurants can sign up, trial, activate, and manage plan state
P0	Restaurant CMS shell	Owners can configure branch, theme, taxes, tables, and service mode
P0	Menu CMS	Categories, subcategories, items, variants, add-ons, schedules
P0	Table sessions and QR	Each table launches the right menu and session context
P0	Shared bucket ordering	Multiple guests collaborate on one live table bucket
P0	Waiter confirmation toggle	Branch can switch between self-service and waiter-confirmed ordering
P0	Service-request buzzer	Customers can request help and staff can resolve with SLA tracking

Priority	Epic	Outcome
P0	Live order queue	Waiters and managers can confirm and monitor orders
P1	Kitchen display	Tickets move through new, preparing, and ready states
P1	Customer payment and bill flows	Pay now, pay later, cashier close, split bill
P1	Core analytics	Conversion, basket size, top items, waiter response time
P1	Promotions and discounts	Coupons, happy hour, category offers
P2	Loyalty and wallets	Repeat-visit retention mechanics
P2	Multi-branch and franchising	Central owner controls and branch comparisons
P2	POS and printer integrations	Operational fit with incumbent restaurant hardware/software
P2	White-label and enterprise controls	SSO, dedicated isolation, custom SLA, advanced exports

gantt

title Rollout roadmap
dateFormat YYYY-MM-DD

section Discovery

Product discovery and pilot interviews :a1, 2026-04-20, 2026-05-03
Architecture and domain finalization :a2, 2026-04-27, 2026-05-10

section Foundation

Tenant onboarding and auth :b1, 2026-05-04, 2026-05-24
Billing core and entitlement engine :b2, 2026-05-11, 2026-05-31
CI/CD, observability, test harness :b3, 2026-05-11, 2026-06-07

section Core product

CMS shell and menu management :c1, 2026-05-18, 2026-06-21
Table management and QR issuance :c2, 2026-05-25, 2026-06-14
Customer PWA and shared bucket :c3, 2026-06-08, 2026-07-12
Waiter confirmation and service requests :c4, 2026-06-22, 2026-07-19

section Operations

Live order queue and kitchen display :d1, 2026-07-06, 2026-08-02
Payment flows and cashier closure :d2, 2026-07-13, 2026-08-09
Core analytics and audit logs :d3, 2026-07-20, 2026-08-16

section Launch

Pilot rollout with selected restaurants :e1, 2026-08-10, 2026-09-06
Hardening, recovery drills, GA readiness :e2, 2026-08-24, 2026-09-20

The milestone gates should be measurable. **Alpha** should mean an owner can onboard a branch, publish a menu, generate table QR codes, and accept an order. **Pilot** should mean at least a handful of real tables can complete the full loop from scan to kitchen to bill with no manual database intervention. **GA** should require recoverable backups, webhook replay safety, role-based audit logs, SLO dashboards, and successful load tests for concurrent table-session collaboration. Target operational metrics should include scan-to-menu under two seconds on median mobile conditions, bucket fanout under one second, order-submit p95 below 500 ms for normal payloads, and service-request acknowledgement inside the restaurant's configured SLA window.

1 24 <https://web.dev/learn/pwa/progressive-web-apps>

<https://web.dev/learn/pwa/progressive-web-apps>

2 5 37 <https://learn.microsoft.com/en-us/azure/azure-sql/database/saas-tenancy-app-design-patterns?view=azuresql>

<https://learn.microsoft.com/en-us/azure/azure-sql/database/saas-tenancy-app-design-patterns?view=azuresql>

3 6 7 16 25 26 28 <https://socket.io/docs/v4/rooms/>

<https://socket.io/docs/v4/rooms/>

4 9 17 <https://stripe.com/billing/pricing>

<https://stripe.com/billing/pricing>

8 29 <https://docs.github.com/articles/getting-started-with-github-actions>

<https://docs.github.com/articles/getting-started-with-github-actions>

10 18 20 38 39 40 <https://render.com/pricing>

<https://render.com/pricing>

11 22 <https://docs.stripe.com/billing/subscriptions/usage-based/meters/configure>

<https://docs.stripe.com/billing/subscriptions/usage-based/meters/configure>

12 <https://www.postgresql.org/docs/current/datatype-json.html>

<https://www.postgresql.org/docs/current/datatype-json.html>

13 33 <https://owasp.org/www-project-api-security/>

<https://owasp.org/www-project-api-security/>

14 https://docs.stripe.com/api/idempotent_requests

https://docs.stripe.com/api/idempotent_requests

15 <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/saas-lens.html>

<https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/saas-lens.html>

19 <https://docs.stripe.com/subscriptions>

<https://docs.stripe.com/subscriptions>

21 27 35 https://commission.europa.eu/law/law-topic/data-protection_en

https://commission.europa.eu/law/law-topic/data-protection_en

23 <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>

30 <https://developer.chrome.com/docs/lighthouse/overview>

<https://developer.chrome.com/docs/lighthouse/overview>

31 <https://www.postgresql.org/docs/current/app-pgdump.html>

<https://www.postgresql.org/docs/current/app-pgdump.html>

32 42 <https://aws.amazon.com/fargate/pricing/>

<https://aws.amazon.com/fargate/pricing/>

34 [PCI-DSS-v4-0-SAQ-A.pdf](#)

https://listings.pcisecuritystandards.org/documents/PCI-DSS-v4-0-SAQ-A.pdf?utm_source=chatgpt.com

36 <https://docs.stripe.com/customer-management>

<https://docs.stripe.com/customer-management>

41 <https://stripe.com/pricing>

<https://stripe.com/pricing>